基于tidevice实现iOS app自动化

一、IOS自动化工具概述

首先需要知道在iOS设备上是谁在执行发送过去的指令，它就是XCUITest，下面介绍它的由来和作用：

XCTest和XCUITest是 iOS 开发中常用的两个测试框架，它们之间存在一定的区别和联系。

```
XCTest：是苹果官方提供的测试框架，可以在 Objective-C 和 Swift 语言中使用。它提供了很多功能强大的测试工具，包括单元测试、性能测试、UI 测试等。使用 XCTest 可以帮助开发者快速编写和运行各种类型的测试用例，并得到详细的测试报告。
XCUITest：是 XCTest 框架的扩展，是苹果官方提供的用于 iOS 应用程序 UI 测试的框架。它支持 Objective-C 和 Swift 语言，并提供了一组 API，用于操作 iOS 应用程序的用户界面元素。使用 XCUITest 可以模拟用户对应用程序的操作，如点击按钮、输入文本、滑动屏幕等，并验证应用程序的行为是否符合预期。
```

从上面可知XCUITest只是一个在iOS设备上执行指令的工具，它并不具备通信能力，所以也就不能和类似appium和wda这些框架进行交互；所以我们还需要一个用于和外部通信的工具，它就是WebDriverAgent，它的作用如下：

```
WebDriverAgent（简称WDA）是一个开源的工具，由 Facebook 提供，用于在 iOS 设备上进行自动化测试。它是基于 WebDriver 协议的实现，通过与客户端的交互来控制 iOS 设备上的应用程序。
WebDriverAgent 的主要功能是提供一个接口，使客户端可以发送各种命令和请求，操作 iOS 设备上的应用程序。这些命令和请求可以包括点击按钮、输入文本、滑动屏幕、获取元素信息等等。客户端可以使用各种编程语言、测试框架或测试工具与 WebDriverAgent 进行通信，以实现自动化测试。
WebDriverAgent 使用 XCTest 框架来运行测试，并且需要通过 Xcode 进行构建和安装到 iOS 设备上。它通过与设备上的 WebDriverAgentRunner 应用程序进行通信来执行测试。
WebDriverAgentRunner 是一个特殊的应用程序，用于启动和管理 WebDriverAgent，以及与客户端进行通信。
WebDriverAgent 功能强大且灵活，可以支持各种类型的自动化测试，如 UI 测试、性能测试、功能测试等。它也提供了丰富的 API，用于与 iOS 应用程序进行交互和验证应用程序的行为。
```

WebDriverAgent是基于XCTest框架的，所以需要使用xcodebuild来启动WebDriverAgent，可以很容易的运行在Mac电脑上；可是我们现在没有Mac电脑，那么需要如何才能在Windows或是Linux上执行IOS自动化测试呢，于是tidevice就出现了，它是由阿里开源的可以实现在Windows和Linux上也能执行IOS的自动化测试。

二、tidevice工具的原理和使用

要使用tidevice完成iOS的自动化测试，首先需要确保手机上已经安装有WebDriverAgent，并且知道bundleId，安装WebDriverAgent教程推荐：https://tester-joe.blog.csdn.net/article/details/133134348

2.1、tidevice的原理

tidevice 的底层原理主要基于libimobiledevice库和usbmux通信协议。libimobiledevice是一个用于与iOS设备进行通信的开源库，而usbmux通信协议则实现了Mac/Windows/Linux与iOS设备服务间的通信。通过模拟xcodebuild与手机进行通信，tidevice向手机发送特定的指令，启动WDA，从而实现了在Linux和Windows上运行iOS自动化的功能。

2.2、tidevice实现的功能

```
获取设备信息
应用安装、卸载、启动、停止、查看应用信息、已安装应用列表
设备截图、获取设备日志
启动 WebDriverAgent （跨平台：Mac、Windows、Linux）
性能数据采集
```

2.3、tidevice的安装

Python环境：Python 3.6+
tidevice 安装
方式一：

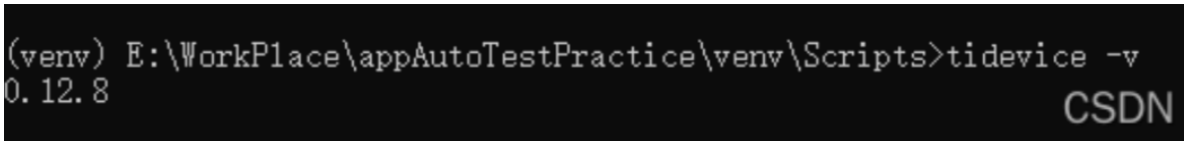
```
pip install -U "tidevice[openssl]" （推荐）
```

方式二

```
pip install -U tidevice （缺少设备配对功能）
```

查看是否安装成功

```
tidevice -v
```



2.4、tidevice的使用

2.4.1、设备管理

1、查看已连接的设备的列表

- **cmd**

```
tidevice list
```

- **python**

```
from tidevice import Usbmux
print(Usbmux().device_list())
```

2、检测设备连接状态

说明：该功能是实时获取，已连接设备、断开设备的状态，所以运行该命令/程序 会挂起阻塞后面的流程，如果不想阻塞后面进程，使用subprocess模块开启子进程完成监测。

- **cmd**

```
tidevice watch
```

```
(venv) E:\WorkPlace\appAutoTestPractice\venv\Scripts>tidevice watch
[I 240131 10:55:51 __main__:228] {'DeviceID': 6,
     'MessageType': 'Attached',
     'Properties': {'ConnectionType': 'USB',
                    'DeviceID': 6,
                    'LocationID': 0,
                    'ProductID': 4776,
                    'SerialNumber': '[redacted]',
                    'UDID': '[redacted]'}
[I 240131 10:56:43 __main__:228] {'DeviceID': 6, 'MessageType': 'Detached'}
[I 240131 10:56:54 __main__:228] {'DeviceID': 7,
     'MessageType': 'Attached',
     'Properties': {'ConnectionType': 'USB',
                    'DeviceID': 7,
                    'LocationID': 0,
                    'ProductID': 4776,
                    'SerialNumber': '[redacted]',
                    'UDID': '[redacted]
```

CSDN @MY测试之路

- **python**

  watch_device()函数返回的是一个生成器对象，所以需要使用循环的方式获取数据，获取的数据是字典类型，所以可以根据字段MessageType的值判断设备的连接情况。

```python
from tidevice import Usbmux
for data in Usbmux().watch_device():
    print(data)
```

3、等待设备连接，只要有就连接就结束监听

- **cmd**
  - 如果已经存在设备连接，检测到后自动结束该命令；如果没有设备连接直到等到有设备连接，再结束该命令

```
tidevice wait-for-device
```

4、等待指定设备连接

- **cmd**
  - 等待指定设备连接，如果指定设备有连接，则结束该命令，否则一直挂起等待指定设备连接
  - 注意：`tidevice -u $UDID watch` 改命令并不具备上面功能，作用还是监测设备的连接情况

```
tidevice -u $UDID wait-for-device
```

5、实时获取设备操作日志

- **cmd**
  - 该命令会实时打印设备的操作日志，类似 `adb logcat` 的作用

```
tidevice -u $UDID syslog
```

- **Python**

```python
from tidevice import Device
device = Device("$UDID")
# 开启 com.apple.syslog_relay 服务 它的作用是用于记录系统日志
socket_proxy = device.start_service("com.apple.syslog_relay")
while True:
    print(socket_proxy.get_socket().recv(1024).decode('utf-8'))
```

6、获取设备信息

- **cmd**
    - 获取设备型号、设备名、系统版本、手机号、序列号、时区、蓝牙地址、Wifi 地址等

```
tidevice info
```

- **Python**

```python
from tidevice import Device
print(Device("$UDID").device_info())
```

7、查看设备电源信息

- **cmd**

```
tidevice info --domain com.apple.mobile.battery --json
```

- **Python**

```python
from tidevice import Device
import json
domain_info = Device("$UDID").device_info("com.apple.mobile.battery")
print(json.dumps(domain_info))
```

8、系统信息

- **cmd**

```
tidevice sysinfo
```

- **Python**

```python
from tidevice import Device
Device("$UDID").instruments.system_info()
```

9、关机

- cmd

```
tidevice shutdown
```

- Python

```python
from tidevice import Device
Device("$UDID").shutdown()
```

10、重启设备

- cmd

```
tidevice reboot
```

- Python

```python
from tidevice import Device
Device("$UDID").reboot()
```

11、截屏保存在当前目录

- cmd

```
tidevice screenshot
```

- Python

```python
from tidevice import Device
filename = "screenshot.jpg"
Device("$UDID").screenshot().convert("RGB").save(filename)
```

12、截屏保存在指定目录

- cmd

```
tidevice screenshot /xxx/xxx/screenshot.png
```

- Python

```python
from tidevice import Device
file_path = "/xxx/xxx/screenshot.png"
Device("udid").screenshot().convert("PNG").save(file_path)
```

13、设备文件管理

- **cmd**

```
tidevice -u $UDID fsync
```

以下是一些常见的文件同步操作及其对应的命令和参数示例：

  - 列出指定路径下的文件: `tidevice fsync ls /path/to/directory`
  - 删除文件: `tidevice fsync rm /path/to/file`
  - 查看文件内容: `tidevice fsync cat /path/to/file`

    - 将文件从设备上拉取到本地: `tidevice fsync pull /path/to/device/file /path/to/local/destination`
    - 将文件推送到设备上: `tidevice fsync push /path/to/local/file /path/to/device/destination`
    - 查看文件或目录的详细信息: `tidevice fsync stat /path/to/file_or_directory`
    - 列出指定路径下的文件和目录树: `tidevice fsync tree /path/to/directory`
- **Python**

```python
from tidevice import Device
# 示例: 获取目录"/"下的文件信息
print(Device("$UDID").sync.listdir_info("/"))
```

14、应用文件管理

- **cmd**

```
tidevice -u $UDID fsync -B bundle_id

该命令需要提供两个参数：command 和 arguments。
command：指定要执行的文件同步操作，如 ls、rm、cat 等。
arguments：根据所选的文件同步操作，提供相应的参数。

以下是一些常见的文件同步操作及其对应的命令和参数示例：
列出指定路径下的文件: tidevice fsync -B com.king.Money ls /path/to/directory
删除文件: tidevice fsync -B com.king.Money rm /path/to/file
查看文件内容: tidevice fsync -B com.king.Money cat /path/to/file
将文件从设备上拉取到本地: tidevice fsync -B com.king.Money pull /path/to/device/file
/path/to/local/destination
将文件推送到设备上: tidevice fsync -B com.king.Money push /path/to/local/file
/path/to/device/destination
查看文件或目录的详细信息: tidevice fsync -B com.king.Money stat
/path/to/file_or_directory
列出指定路径下的文件和目录树: tidevice fsync -B com.king.Money tree
/path/to/directory
```

- **Python**

```python
from tidevice import Device
# 示例：获取目录"/"下的文件信息
print(Device("$UDID").app_sync(bundle_id).listdir_info("/")
```

2.4.2、应用管理

1、安装应用

- **cmd**

    - `file_or_url`：表示本地的 `.ipa` 文件地址路径或 `.ipa` 包的下载路径
    - 使用 `url` 安装的底层原理：先在本地生成一个临时空的 `.ipa` 文件，路径为：`C:\Users\Admin\AppData\Local\Temp\tmp42jr_gnw\_tmp.ipa`；然后请求下载链接，一边下载一边把下载的内容写入到 `_tmp.ipa` 文件中，最后使用 `_tmp.ipa` 安装。

```
tidevice --udid $UDID install file_or_url
```

或

```
tidevice -u $UDID install file_or_url
```

- **Python**

```
from tidevice import Device
device = Device("$UDID")
device.app_install(file_or_url)
```

2、卸载应用

- **cmd**
  - `bundle_id`：类比于 Android 的包名

```
tidevice --udid $UDID uninstall bundle_id
```

- **Python**

```
from tidevice import Device
device = Device("$UDID")
device.app_uninstall(bundle_id)
```

3、启动应用

cmd

- 注意：该命令执行每次都是冷启动，也就是如果应用是打开的会先把应用 kill 掉，然后再启动。

```
# 作用：冷启动iOS设备上指定Bundle ID的应用
# 特点：无论应用当前是否运行，都会先终止已有进程再重新启动（冷启动）
# 参数说明：
#   --udid $UDID : 指定要操作的iOS设备唯一标识符（将$UDID替换为实际设备UDID）
#   launch       : tidevice的启动应用指令
#   bundle_id    : 要启动的应用的Bundle ID（类比Android的包名，需替换为实际值）
# 使用示例：tidevice --udid 12345678-ABCD-EFGH-IJKL-1234567890AB launch
com.apple.mobilesafari
tidevice --udid $UDID launch bundle_id
```

Python

```
from tidevice import Device
# 注意此方式是强制的冷启动，如果应用已打开则先kill旧进程，然后再启动
pid = Device("$UDID").app_start(bundle_id)
```

如果想实现：

- 已启动且最上层应用，则保持不变；
- 已启动且退到后台，把应用拉起；

- 如果未启动，冷启动。

则需要改变源码中 `app_launch()` 函数下的 `KillExisting` 变量，将其改为 `False`。但是请注意，设置 `KillExisting` 为 `False`，调用 60 次以上，instruments 服务将崩溃，原因是：多次启动相同的应用程序实例并且不终止旧的进程会导致系统资源耗尽或冲突。这可能会导致 "instruments" 服务无法正常运行，并最终崩溃。

```
options = {
    # don't suspend the process after starting it
    "StartSuspendedKey": 0,
    # Note: set KillExisting to False and call 60+ times, instruments service will crash
    "KillExisting": True,        将其改为False即可实现上面的功能，请注意上面的提示。
    # I donot know much about it, when set to True, app will have a pid, but not show up
    # "ActivateSuspended": False,
}                                                                    CSDN @MY测试之路
```

4、关闭应用

- **cmd**
  - `pid_or_name`：表示进程 ID 或包名（bundle_id）

```
tidevice --udid $UDID kill pid_or_name
```

- **Python**

```
from tidevice import Device
Device("$UDID").app_stop(pid_or_name)
```

5、查看已安装应用列表

- **cmd**

```
# 查看指定设备的应用列表
tidevice --udid $UDID applist
# 只有一个设备连接的情况，可直接查看
tidevice applist
```

- **Python**

```
from tidevice import Device
device = Device("$UDID")
# 结果会包含设备上全部的应用信息，包括用户应用('type': 'User')，系统应用('type':
'system')，插件('type': 'PluginKit')
apps = device.connect_instruments().app_list()
```

6、获取指定应用详细应用信息

- **cmd**
  - 可以获取：详细信息，如版本、构建版本、安装日期、所需最低操作系统版本等

```
tidevice appinfo bundle_id
```

- **Python**

```python
from tidevice import Device
device = Device("$UDID")
app_info = device.installation.lookup(bundle_id)
print(app_info)
```

2.4.3、执行自动化测试

1、启动 WebDriverAgent 服务

- **cmd**
    - `WebDriverAgent_bundle_id`：表示 WebDriverAgent 的 bundle_id

```bash
tidevice wdaproxy -B WebDriverAgent_bundle_id --port 8100
# 指定设备启动WebDriverAgent
tidevice -u $UDID wdaproxy -B WebDriverAgent_bundle_id --port 8100
```

    - 命令详解：
        - `tidevice -u $UDID`：连接到指定 UDID 的 iOS 设备，UDID 是设备的唯一标识
        - `wdaproxy -B WebDriverAgent_bundle_id`：启动 WebDriverAgent，并将其绑定到指定的 WebDriverAgent_bundle_id。WebDriverAgent 是一个允许远程控制 iOS 设备的工具，它需要一个唯一的标识符来与设备进行通信
        - `--port 8100`：指定将 WebDriverAgent 代理到本地的 8100 端口。这样，你可以通过访问本地的 8100 端口来控制设备上的 WebDriverAgent
    - 启动后通过访问`http://127.0.1:8100/status`，检测 WebDriverAgent 服务是否成功启动。

2.4.4、fps 数据采集

- **cmd**

```bash
tidevice dumpfps
```

- **Python**

```python
# 导入tidevice库中的Device类，该类是操作iOS设备的核心类
from tidevice import Device

# 初始化Device对象，建立与指定iOS设备的连接
# 参数说明：
#   "$UDID"：需要替换为你实际的设备唯一标识符（iOS设备的UDID）
#   示例：device = Device("12345678-ABCD-EFGH-IJKL-1234567890AB")
device = Device("$UDID")

# 循环采集并输出iOS设备的FPS（帧率）相关数据
# 核心逻辑拆解：
# 1. device.connect_instruments()：建立与设备的instruments服务连接（iOS性能数据采集的核心通道）
```

```
# 2. iter_opengldata()：迭代获取设备的OpenGL渲染数据（包含FPS、渲染耗时等关键性能指标）
# 3. for循环：持续读取每帧的性能数据并打印
for data in device.connect_instruments().iter_opengldata():
    # 打印每一次采集到的FPS相关原始数据
    # 数据格式示例：{'time': 1710000000.123, 'fps': 59.8, 'present_time': 16.7, ...}
    print(data)
```